

Lab Exercise: Refactoring a Real-World C# Application Using GoF Design Patterns

Lab Title

Refactoring an E-Commerce Order Processing System Using GoF Design Patterns

Objective

In this lab, you will learn how to:

- Identify bad design/code smells
 - Understand why the code becomes difficult to maintain
 - Refactor step-by-step using GoF Design Patterns
 - Apply SOLID principles naturally
 - Improve extensibility and maintainability
-

Patterns Covered

Design Pattern	Purpose
Strategy Pattern	Remove complex conditional logic
Factory Method Pattern	Centralize object creation
Observer Pattern	Handle notifications cleanly
Decorator Pattern	Add optional features dynamically

Realistic Scenario

You are developing an **E-Commerce Order Processing System**.

The system:

- Calculates payment charges
- Sends notifications
- Applies optional premium services

Initially, the code was written quickly and now has:

- Huge `if-else` blocks
- Duplicate logic
- Tight coupling
- Difficult maintenance

Your task:

Refactor the application step-by-step using GoF patterns.

Initial BAD CODE

Problem Statement

The application supports:

- Credit Card payment
- UPI payment
- PayPal payment

It also:

- Sends Email/SMS notifications
- Adds Gift Wrap and Insurance

Everything is written in one class.

Step 1 — Create the BAD Implementation

Create Console Application

```
dotnet new console -n EcommerceBadDesign
```

Replace Program.cs

```
using System;

class OrderProcessor
{
    public void ProcessOrder(string paymentType,
                             double amount,
                             bool giftWrap,
                             bool insurance,
                             string notificationType)
    {
```

```

        double finalAmount = amount;

        // Optional Features
        if (giftWrap)
        {
            finalAmount += 100;
            Console.WriteLine("Gift wrap added");
        }

        if (insurance)
        {
            finalAmount += 200;
            Console.WriteLine("Insurance added");
        }

        // Payment Processing
        if (paymentType == "CreditCard")
        {
            Console.WriteLine("Processing Credit Card Payment");
            finalAmount += finalAmount * 0.02;
        }
        else if (paymentType == "UPI")
        {
            Console.WriteLine("Processing UPI Payment");
        }
        else if (paymentType == "PayPal")
        {
            Console.WriteLine("Processing PayPal Payment");
            finalAmount += 50;
        }

        Console.WriteLine($"Final Amount: {finalAmount}");

        // Notifications
        if (notificationType == "Email")
        {
            Console.WriteLine("Sending Email Notification");
        }
        else if (notificationType == "SMS")
        {
            Console.WriteLine("Sending SMS Notification");
        }
    }
}

class Program
{
    static void Main()
    {
        OrderProcessor processor = new OrderProcessor();

        processor.ProcessOrder(
            paymentType: "CreditCard",
            amount: 1000,
            giftWrap: true,
            insurance: true,
            notificationType: "Email");
    }
}

```

Step 2 — Execute and Observe Problems

Run the application.

```
dotnet run
```

Step 3 — Identify BAD DESIGN Problems

Problem 1 — Too Many Responsibilities

`OrderProcessor` handles:

- Payment
- Notifications
- Feature additions
- Pricing

Violates:

- Single Responsibility Principle
-

Problem 2 — Massive if-else Conditions

```
if(paymentType == "CreditCard")
```

Problems:

- Hard to maintain
- Difficult to test
- New payment requires code modification

Violates:

- Open Closed Principle
-

Problem 3 — Tight Coupling

`OrderProcessor` directly depends on:

- Email
- SMS
- CreditCard

- PayPal

Changing one thing affects everything.

Problem 4 — Feature Explosion

Adding:

- Coupon
- Priority delivery
- Loyalty points

Will make code worse.

REFACTORING PHASE

Part 1 — Apply Strategy Pattern

Why Strategy Pattern?

We want to remove this:

```
if(paymentType == "CreditCard")
```

Different payment methods are different behaviors.

Perfect use case for:

Strategy Pattern

Step 4 — Create Payment Strategy

Create Interface

```
interface IPaymentStrategy
{
    double Pay(double amount);
}
```

```
}
```

Step 5 — Create Concrete Strategies

CreditCardPayment.cs

```
class CreditCardPayment : IPaymentStrategy
{
    public double Pay(double amount)
    {
        Console.WriteLine("Processing Credit Card Payment");
        return amount + (amount * 0.02);
    }
}
```

UpiPayment.cs

```
class UpiPayment : IPaymentStrategy
{
    public double Pay(double amount)
    {
        Console.WriteLine("Processing UPI Payment");
        return amount;
    }
}
```

PaypalPayment.cs

```
class PaypalPayment : IPaymentStrategy
{
    public double Pay(double amount)
    {
        Console.WriteLine("Processing PayPal Payment");
        return amount + 50;
    }
}
```

Step 6 — Refactor OrderProcessor

```
class OrderProcessor
{
    private readonly IPaymentStrategy _paymentStrategy;

    public OrderProcessor(IPaymentStrategy paymentStrategy)
    {
        _paymentStrategy = paymentStrategy;
    }

    public void ProcessOrder(double amount)
```

```
    {  
        double finalAmount = _paymentStrategy.Pay(amount);  
        Console.WriteLine($"Final Amount: {finalAmount}");  
    }  
}
```

Step 7 — Test Strategy Pattern

```
class Program  
{  
    static void Main()  
    {  
        IPaymentStrategy strategy =  
            new CreditCardPayment();  
  
        OrderProcessor processor =  
            new OrderProcessor(strategy);  
  
        processor.ProcessOrder(1000);  
    }  
}
```

What Improved?

Before	After
if-else blocks	Polymorphism
Hard-coded behavior	Flexible behavior
Difficult extension	Easy extension

Add New Payment Easily

BitcoinPayment.cs

```
class BitcoinPayment : IPaymentStrategy  
{  
    public double Pay(double amount)  
    {  
        Console.WriteLine("Processing Bitcoin Payment");  
        return amount + 10;  
    }  
}
```

NO modification to OrderProcessor.

Part 2 — Apply Factory Method Pattern

Why Factory?

Now object creation is becoming messy:

```
IPaymentStrategy strategy =  
    new CreditCardPayment();
```

Client should not know concrete classes.

Step 8 — Create Factory

```
class PaymentFactory  
{  
    public static IPaymentStrategy Create(string paymentType)  
    {  
        return paymentType switch  
        {  
            "CreditCard" => new CreditCardPayment(),  
            "UPI" => new UpiPayment(),  
            "PayPal" => new PaypalPayment(),  
            _ => throw new Exception("Invalid Payment Type")  
        };  
    }  
}
```

Step 9 — Refactor Main

```
class Program  
{  
    static void Main()  
    {  
        var strategy =  
            PaymentFactory.Create("UPI");  
  
        var processor =  
            new OrderProcessor(strategy);  
  
        processor.ProcessOrder(1000);  
    }  
}
```

What Improved?

Before	After
Client knows concrete classes	Factory hides creation
Scattered object creation	Centralized creation
Hard to manage	Easy maintenance

Part 3 — Apply Observer Pattern

Problem

Notification logic still tightly coupled.

We want:

- Email
- SMS
- WhatsApp
- Push Notification

Without modifying order processor.

Step 10 — Create Observer Interface

```
interface IOrderObserver
{
    void Update(string message);
}
```

Step 11 — Create Observers

EmailNotifier.cs

```
class EmailNotifier : IOrderObserver
{
    public void Update(string message)
    {
        Console.WriteLine($"EMAIL: {message}");
    }
}
```

SmsNotifier.cs

```
class SmsNotifier : IOrderObserver
{
    public void Update(string message)
    {
        Console.WriteLine($"SMS: {message}");
    }
}
```

Step 12 — Add Subject Logic

```
class OrderProcessor
{
    private readonly IPaymentStrategy _paymentStrategy;

    private readonly List<IOrderObserver> _observers =
        new List<IOrderObserver>();

    public OrderProcessor(IPaymentStrategy paymentStrategy)
    {
        _paymentStrategy = paymentStrategy;
    }

    public void AddObserver(IOrderObserver observer)
    {
        _observers.Add(observer);
    }

    private void Notify(string message)
    {
        foreach (var observer in _observers)
        {
            observer.Update(message);
        }
    }

    public void ProcessOrder(double amount)
    {
        double finalAmount =
            _paymentStrategy.Pay(amount);

        Console.WriteLine($"Final Amount: {finalAmount}");

        Notify("Order Processed Successfully");
    }
}
```

Step 13 — Test Observer Pattern

```
class Program
{
    static void Main()
    {
        var strategy =
            PaymentFactory.Create("CreditCard");

        var processor =
            new OrderProcessor(strategy);

        processor.AddObserver(new EmailNotifier());
        processor.AddObserver(new SmsNotifier());

        processor.ProcessOrder(1000);
    }
}
```

What Improved?

Before	After
Hardcoded notifications	Dynamic subscribers
Tight coupling	Loose coupling
Difficult extension	Easy to add channels

Part 4 — Apply Decorator Pattern

Problem

Optional features:

- Gift Wrap
- Insurance
- Priority Delivery

Currently implemented using booleans.

This causes:

- Parameter explosion
 - Endless conditions
-

Step 14 — Create Component Interface

```
interface IOrderCost
{
    double GetCost();
}
```

Step 15 — Base Order

```
class BaseOrder : IOrderCost
{
    private readonly double _amount;

    public BaseOrder(double amount)
    {
        _amount = amount;
    }

    public double GetCost()
    {
        return _amount;
    }
}
```

Step 16 — Create Abstract Decorator

```
abstract class OrderDecorator : IOrderCost
{
    protected readonly IOrderCost _order;

    protected OrderDecorator(IOrderCost order)
    {
        _order = order;
    }

    public virtual double GetCost()
    {
        return _order.GetCost();
    }
}
```

Step 17 — Create Concrete Decorators

GiftWrapDecorator.cs

```
class GiftWrapDecorator : OrderDecorator
{
    public GiftWrapDecorator(IOrderCost order)
        : base(order)
    {
    }
}
```

```

    {
    }

    public override double GetCost()
    {
        return base.GetCost() + 100;
    }
}

```

InsuranceDecorator.cs

```

class InsuranceDecorator : OrderDecorator
{
    public InsuranceDecorator(IOrderCost order)
        : base(order)
    {
    }

    public override double GetCost()
    {
        return base.GetCost() + 200;
    }
}

```

Step 18 — Test Decorator Pattern

```

class Program
{
    static void Main()
    {
        IOrderCost order =
            new BaseOrder(1000);

        order =
            new GiftWrapDecorator(order);

        order =
            new InsuranceDecorator(order);

        Console.WriteLine(
            $"Total Cost: {order.GetCost()}");
    }
}

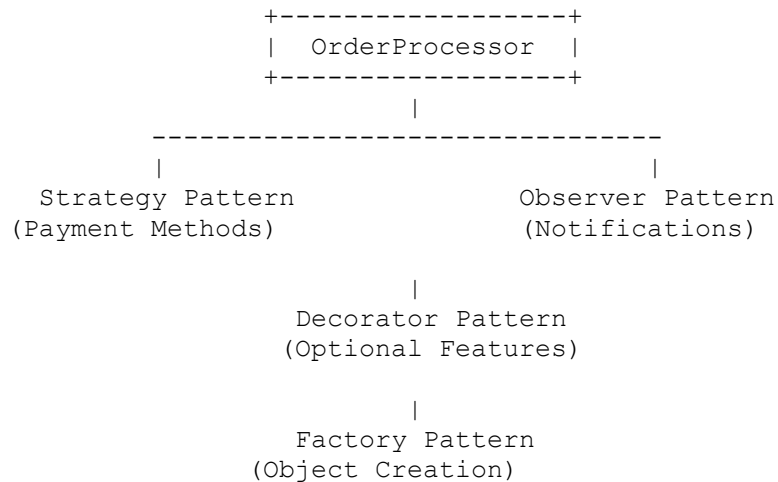
```

What Improved?

Before	After
Boolean parameters	Dynamic composition
Feature explosion	Flexible layering

Before	After
Hard to extend	Easy extension

Final Architecture



Final Learning Summary

Problem	Pattern Used
Huge if-else blocks	Strategy
Complex object creation	Factory
Tight notification coupling	Observer
Optional feature explosion	Decorator

Practice Tasks

Task 1

Add:

- Crypto payment
- NetBanking payment

Using Strategy Pattern.

Task 2

Add:

- Push notification
- WhatsApp notification

Using Observer Pattern.

Task 3

Add:

- Coupon discount
- Express delivery

Using Decorator Pattern.

Advanced Challenge

Refactor the entire solution using:

- Dependency Injection
 - Interfaces everywhere
 - ASP.NET Core Web API
-

Expected Learning Outcome

After this lab, students should be able to:

- Recognize bad object-oriented design
 - Understand when to apply GoF patterns
 - Refactor incrementally
 - Reduce coupling
 - Improve maintainability
 - Build extensible enterprise applications
-

Recommended Next Patterns

Next GoF patterns to practice:

1. Adapter Pattern
2. Facade Pattern
3. Command Pattern
4. Builder Pattern
5. Template Method Pattern
6. State Pattern
7. Chain of Responsibility Pattern
8. Mediator Pattern